

Contexto del Proyecto: Enfoque en ORM (Sequelize)

Este documento resume la arquitectura del modelo de datos de la aplicación y la gestión de la base de datos a través de Sequelize ORM. Útil para iniciar contexto con otros agentes.

1. Stack Tecnológico del ORM

- **Base de Datos:** PostgreSQL (ejecutada en contenedor Docker).
- **Motor ORM:** Sequelize ORM (v6) integrado con ES Modules nativos de Node.js.
- **Dialecto:** postgres con configuración centralizada en `app/src/config/database.cjs`.
- **Configuración del CLI:** `.sequelizerc` mapea las carpetas de modelos, migraciones y seeders directamente dentro de la subcarpeta `app/src/`.

2. Arquitectura de Modelos (`app/src/models/`)

Todos los modelos están implementados bajo el patrón de funciones fábrica para recibir la instancia de conexión única de Sequelize y los tipos de datos lógicos (DataTypes).

• Barra (`barra.js`):

Representa la tabla física **barras**. Contiene `numero_barra` (INTEGER, único, no nulo) y `sector` (STRING(100), no nulo). Tiene un hook `beforeSave` para formatear el valor de `sector` a letras mayúsculas antes de persistir.

• Bebida (`bebida.js`):

Representa la tabla física **bebidas**. Contiene `nombre` (STRING(200), único, no nulo), `precio` (DECIMAL(15,2), no nulo, default 0) y `descripcion` (TEXT, permite nulos). Nota: el atributo `stock` está comentado en el modelo en JavaScript para ejercicios.

• Venta (`venta.js`):

Representa la tabla física **ventas**. Contiene `numero_venta` (INTEGER, único, no nulo), `fecha` (DATEONLY, no nulo), `hora` (TIME, no nulo) y `total` (DECIMAL(15,2), no nulo, default 0).

• DetalleVenta (`detalleventa.js`):

Representa la tabla física **detalle_ventas**. Contiene `venta_id` (INTEGER, no nulo), `bebida_id` (INTEGER, no nulo), `barra_id` (INTEGER, no nulo), `cantidad` (INTEGER, no nulo, default 1) y `subtotal` (DECIMAL(15,2), no nulo, default 0).

3. Relaciones y Asociaciones (app/src/models/index.js)

Las relaciones lógicas de clave foránea están centralizadas en el archivo indexador mediante asociaciones nativas de Sequelize:

1. Relación Venta <-> DetalleVenta (Uno a Muchos):

```
Venta.hasMany(DetalleVenta, { foreignKey: 'venta_id', as: 'Detalle' })
```

```
DetalleVenta.belongsTo(Venta, { foreignKey: 'venta_id', as: 'Venta' })
```

2. Relación Bebida <-> DetalleVenta (Uno a Muchos):

```
Bebida.hasMany(DetalleVenta, { foreignKey: 'bebida_id', as: 'Detalle' })
```

```
DetalleVenta.belongsTo(Bebida, { foreignKey: 'bebida_id', as: 'Bebida' })
```

3. Relación Barra <-> DetalleVenta (Uno a Muchos):

```
Barra.hasMany(DetalleVenta, { foreignKey: 'barra_id', as: 'Detalle' })
```

```
DetalleVenta.belongsTo(Barra, { foreignKey: 'barra_id', as: 'Barra' })
```

4. Gestión del Esquema: Migraciones (app/src/migrations/)

La evolución de la base de datos se maneja mediante scripts ordenados cronológicamente por marcas de tiempo:

• **Migración Inicial (inicial.js):**

Utiliza `queryInterface.createTable` para generar la estructura física de las tablas en PostgreSQL. Define restricciones (`allowNull: false`, `unique: true`), claves primarias autoincrementales, y claves foráneas físicas (referencias a nivel de base de datos) con reglas `onDelete: 'RESTRICT'`. Su método `down` revierte la creación en orden inverso.

• **Migración Incremental (agregar-stock-bebidas.js):**

Utiliza `queryInterface.addColumn` en el método `up` para añadir la columna `stock` (tipo `Sequelize.INTEGER`, no nula, default 0) a la tabla física `bebidas`. Usa `queryInterface.removeColumn` en el método `down` para revertir.